

Quantum Kingdom

Game Design Document (GDD)

Autora:
Olaia Picabea Sainz

Indice

1. Especificaciones.....	3
1.1. Especificaciones Técnicas.....	3
2. Game Play.....	3
3. Game Play Outline.....	4
4. Características Clave.....	4
5. Diseño de Juego.....	5
5.1. Guías del Diseño de Juego.....	5
5.2. Definiciones del Diseño de Juego.....	5
5.3. Diagrama de Flujo.....	6
5.4. Definición del Jugador.....	7
5.4.1. Propiedades del Jugador.....	7
6. Interfaz de Usuario (UI).....	9
7. Diferencias de implementación (apartado grupal).....	10
8. Aporte individual.....	12
8.1. Pasar blueprints a clases de C++.....	12
8.2. Mejorar aspectos visuales.....	12
8.3. Añadir socket de arma en la muñeca del player.....	12
8.4. Nuevos enemigos + IA.....	13
8.5. Mejorar presencia.....	13
8.6. Diferencias con el GDD original.....	13

1. Especificaciones

Game name: Quantum Kingdom

Genre: Shooter en realidad virtual (VR)

Game Elements:

Genre elements are the basic activities the player will be doing for fun.

Las mecánicas del juego incluyen disparar tanto a dianas como a enemigos, recargar el arma en caso de que lo requiera, cubrirse en zonas específicas, apuntar con el arma, usar power-ups que permiten tener munición infinita durante un tiempo limitado, recoger armas del suelo o de enemigos y realizar ataques cuerpo a cuerpo. En cuanto al movimiento, el jugador puede andar, correr y saltar.

Player: 1

The number of players that can play the game at once

1.1. Especificaciones Técnicas

Technical Form: 3D

Basically there are 2D graphics (flat) and 3D graphics (form)

View: Primera persona

Camera view from which the player will experience the game

Platform: Android

Language: C++, Unreal Engine

Device: gafas de realidad virtual

2. Game Play

Use the game play section to create a descriptive paragraph about how the game is played. You want the use to imagine they are actually playing the game. Try not to use generic (i.e., broad, non-descriptive) terms when writing about the game play. For example, few readers want to hear statements such as, "enemy_1 will have more hit points than enemy_2." Instead, it's better to make statements like, "The Lazarus Fighter has more armour than the Apollo Fighter."

El personaje principal, Kael Ironhart, es un prisionero que está siendo entrenado para misiones militares. Un día se despierta como todos los días en su nave espacial. Le dicen que su misión será volver al pasado, a luchar y así cambiar el futuro (hay que cambiar el pasado porque la tierra fue invadida por extraterrestres).

El jugador empezará en la nave, donde se le avisará por megafonía que vaya a la sala de entrenamiento. En la sala encontrará diferentes armas y dianas. El usuario tendrá que entrenar y familiarizarse con los objetos. Pasados un par de minutos sonará un audio donde se le pedirá que acuda a la sala de misiones, donde verá un portal que le llevará al mundo medieval.

Una vez en la arena, se le comunicará que si consigue matar a los monstruos se le otorgará la libertad y una vida digna. Cuando el juego empiece, aparecerán monstruos que tendrá que matar. Si el monstruo ataca al usuario, este perderá 1 vida. El jugador no tendrá un timer visible, pero internamente, lo habrá. Ese timer calculará 4 minutos desde que empieza el juego y *spawneará* enemigos por la arena. Cuando ese tiempo termine, aparecerá un último monstruo, mucho mayor, con muchas más vidas y que causa más daño al usuario. Si el usuario consigue superar los retos, se ganará la partida, si no, morirá.

3. Game Play Outline

This outline will vary depending on the type of game.

- Opening the game application

Jugar, que inicia la partida; Ajustes, donde es posible modificar parámetros como el volumen u otras configuraciones; y Salir, que permite cerrar el juego y volver al sistema.

- Game options

No hay game options de momento.

- Story synopsis

En un futuro donde la Tierra ha sido destruida por extraterrestres, los humanos descubren que la invasión pudo evitarse eliminando a unos monstruos medievales en el pasado. Kael Ironhart, un prisionero obligado a unirse a la misión, para ello realiza un entrenamiento y viaja a través de un portal temporal para enfrentarse a esas criaturas con el fin de reescribir el futuro de la humanidad.

- Modes

Modo entrenamiento en la nave, modo supervivencia en la arena y modo contrarreloj 4 minutos de oleadas de enemigos.

- Game elements

Disparar, recargar, cubrirse, apuntar, power ups, recoger armas del suelo, andar, correr, saltar.

- Game levels

Tres niveles: un primer nivel que es la zona de entrenamiento dentro de la nave, un segundo nivel que es las hordas de enemigos dentro de la arena y un tercer nivel que es el final boss.

- Player's controls

Andar con el joystick izquierdo, correr apretar joystick izquierdo, saltar presionando el A derecho. Apuntar con el grip derecho, y disparar con el trigger derecho. Para cambiar la posición de la cámara el joystick derecho.

- Winning

El jugador ganará si sobrevive a las oleadas de enemigos durante 4 minutos y además consigue vencer al boss final que tiene mucha más vida.

- Losing

El jugador perderá si muere en algún momento en la lucha en la arena.

- End

El final será la libertad del jugador que es prisionero.

- Why is all this fun?

Es divertido porque se juega con un mundo futurista y con un mundo medieval lo que conlleva diferentes tipos de armas, además, el juego tiene un fin que es la libertad y ganar y a la vez se puede morir lo que implica un reto al jugador. Hay power ups que agregan cambios en las mecánicas para hacerlo aún más divertido. Hay distintos niveles de enemigos lo que aumenta la dificultad y el reto.

4. Características Clave

Key features are a list of game elements that are attractive to the player.

El juego ofrece un combate VR inmersivo que combina armas avanzadas con ataques cuerpo a cuerpo, acompañado de un sistema de viajes en el tiempo que permite enfrentar monstruos medievales para

alterar el futuro. El jugador dispone de un amplio arsenal que incluye armas futuristas, power-ups y equipo recogido del entorno, todo reforzado por un entrenamiento militar de alta tecnología previo a la misión. La acción se desarrolla con movimiento libre (andar, correr y saltar), dentro de escenarios medievales reinterpretados, llenos de criaturas hostiles y zonas de cobertura estratégicas. La experiencia se completa con una progresión narrativa centrada en Kael Ironhart y su lucha por obtener la libertad.

5. Diseño de Juego

This document describes how GameObjects behave, how they're controlled and their properties. This is often referred to as the "mechanics" of the game. This documentation is primarily concerned with the game itself. This part of the document is meant to be modular, meaning you could have several different Game Design Documents attached to the Concept Document.

5.1. Guías del Diseño de Juego

This is an important statement about any creative restrictions that need to be considered and includes brief statements about the general (i.e., overall) goal of the design.

La meta principal del juego gira en torno a la creación de un entorno virtual e inmersivo de combate, combinando tecnología futurista con una ambientación medieval para ofrecer un shooter vistoso y original. El juego tiene un ritmo rápido de combate, objetivos claros y controles intuitivos para que los jugadores se sientan completamente inmersos en el rol de Kael Ironhart.

Durante el desarrollo, muchas restricciones de diseño han de ser consideradas. Como el juego está diseñado para dispositivos VR, la comodidad del jugador es prioritaria. Las mecánicas de movimiento como correr, saltar y girar se han planteado de manera que minimicen el *"motion sickness"* y mantengan una experiencia de juego suave y comfortable. Los elementos de la interfaz seguirán esta filosofía, integrándose de manera simple y poco intrusiva, para preservar la inmersión e incluso fomentarla.

El gameplay, si bien se plantea como dinámico y frenético, debe mantenerse fácil de entender, con una progresión clara y precisa desde el tutorial hasta la arena y el jefe final. Los diseños de enemigos, armas, power-ups, y entornos deben reforzar el contraste entre elementos futuristas y medievales, para preservar la originalidad del concepto principal. Todas las mecánicas deben, además, satisfacer y actuar junto al loop principal de juego, creando una retroalimentación entre mecánicas y dinámicas de juego. De esta manera, se logrará una experiencia satisfactoria al sobrevivir oleadas de enemigos, perfeccionar la destreza en el combate, y, sobre todo, obtener el objetivo final del jugador de conseguir la libertad y cambiar el futuro.

5.2. Definiciones del Diseño de Juego

This section established the definition of the game play. Definitions should include how a player wins, loses, transitions between levels, and the main focus of the gameplay.

El foco principal del gameplay es ofrecer un combate VR inmersivo, donde el jugador debe sobrevivir a oleadas de enemigos usando armas futuristas y los elementos del escenario a su favor. El juego combina disparos, gestión de recursos, y elementos de supervivencia junto con una narrativa centrada en los viajes en el tiempo y la lucha por alterar el futuro de la humanidad.

El gameplay comienza en una nave, donde el jugador completa una secuencia de entrenamiento para aprender los controles y las mecánicas. Después del entrenamiento, el jugador entra por un portal y aparece en una arena medieval. Es dicha arena donde sucederá la acción principal del juego, y donde el jugador pasará la mayor parte de su tiempo.

La progresión entre niveles es lineal. El jugador avanza a la vez que combate en la arena a partir de unos portales temporales, y finalmente se enfrenta a un poderoso jefe tras sobrevivir numerosas oleadas de enemigos. Cada escenario introduce nuevos retos mientras se mantienen las mecánicas

que conforman el loop principal.

- El jugador gana al sobrevivir todas las oleadas y al derrotar al boss final. Al ganar, el jugador satisface a la narrativa principal de salvar el futuro y cambiarlo
- El jugador pierde cuando todos sus puntos de vida se reducen a 0. Tras esto, aparecerá una pantalla de Game Over con opción de volver al menú principal.

5.3 Diagrama de Flujo

The game flowchart provides a visual of how the different game elements and their properties interact. Game flowcharts should represent Objects, Properties, and Actions present in the game. Each of these items should have a number reference to where they exist within the game mechanics document.

· Menu

- El juego empieza con una pantalla de inicio con el nombre del juego.
- Botón de “Jugar”
- Botón de “Ajustes” (para ajustar volumen,)
- Botón de “Salir” para cerrar el juego.

· Synopsis

- La Tierra fue destruida por extraterrestres y los sobrevivientes tuvieron que escapar a naves espaciales. Después de que los mejores investigadores lo analizaran, se dieron cuenta que matar a unos monstruos de la era medieval hubiera solucionado el problema, por tanto, desarrollaron un portal para viajar al pasado, para que los militares maten a esos monstruos y así revertir la invasión extraterrestre y cambiar el futuro. Kael Ironhart, prisionero por intentar matar al director de la nave, es obligado a prepararse para la misión militar que lo cambiará todo. Su entrenamiento le enseñará a manejar armas avanzadas que luego utilizará para matar a los monstruos medievales. El trato es fácil, si los mata, consigue la libertad, si no, muere.

· Game Play

- El jugador aparece en la nave espacial, dentro de su celda.
- Una voz por megafonía le pide que acuda a la sala de entrenamiento.
- En dicha sala, el jugador podrá coger armas y disparar a dianas para familiarizarse con los controles.
- En 3 minutos, otra voz por megafonía le pedirá que acuda a la sala de misiones, donde encontrará un portal que tendrá que cruzar.
- Una vez en la arena, oirá una voz desde su casco, donde le indicará que si consigue matar a los monstruos, se le otorgará la libertad.
- Empiezan a aparecer monstruos por la arena que el jugador tendrá que matar.
- Si el monstruo ataca al jugador, éste perderá 1 vida.
- El monstruo tendrá 2 vidas.
- Cuando pasen 4 minutos aparecerá un monstruo final, que tendrá 10 vidas y en cada golpe, le quitará 2 vidas al usuario.
- Si el jugador mata al último monstruo, ganará el juego, y si no, perderá.

· Player Control

- Caminar: Joystick izquierdo.
- Correr: Igual que caminar pero presionando el joystick.
- Saltar: Botón A del controlador derecho
- Apuntar: Grip derecho
- Disparar: Trigger derecho
- Girar visión: Joystick derecho

· Game Over (Winning and Losing)

- Se ganará la partida si el jugador consigue derrotar al último monstruo. Aparecerá una pantalla de victoria.
- Se perderá si el usuario pierde todas sus vidas. Aparecerá una pantalla de game over. En esa pantalla habrá un botón para volver a la pantalla de inicio.

5.4. Definición del Jugador

Como se explica en el apartado 1., el jugador encarna a Kael Ironhart, un prisionero sometido a entrenamiento militar en una nave futurista. Por ello, las propiedades que el jugador manejará estarán centradas alrededor de esa misma idea.

Al inicio, mencionado en el apartado 5.3., el jugador interactuará inicialmente con una arma futurista que contará con propiedades de daño y capacidad máxima de munición cargada. También, durante el combate, los enemigos pueden soltar armas medievales que el jugador puede recoger y utilizar. Este sistema resulta clave, ya que cuando el arma inicial se queda sin munición, el jugador dependerá exclusivamente de las armas obtenidas en la arena.

Debido a esto, las interacciones principales que el usuario podrá llevar a cabo son acciones como agarre, disparo y movimiento, todo enfocado al combate en VR.

5.4.1. Propiedades del Jugador

Las propiedades del jugador serán las siguientes:

- **Vida**

Implementa la cantidad máxima de daño que puede recibir el jugador. En cuanto al feedback, al recibir daño y bajar la vida restante, se reproducirá un sonido de golpe para que el usuario entienda que ha recibido daño y para la cantidad restante (a parte de una barra de vida que se explicará más adelante) se teñirá la pantalla de rojo y se reducirá el ruido ambiente junto a un sonido específico (latido de corazón o pitido) cuando esté cercano a la muerte. Si la vida llega a cero, el jugador muere con un sonido de fin de partida, teniendo que volver a empezar.

- **Resistencia**

La capacidad del usuario de aguantar el daño que recibe. Esto implica que, al recibir daño, una parte del mismo queda reducida por absorción. Facilitando el equilibrio numérico del daño de las armas. Además, puede ser modificable en función de la vida restante, lo que ayuda a que una vez entras en el estado de poca vida, puedas recibir más golpes sin morir. Esto es efectivo en bajar el nivel de dificultad y estrés del usuario en situaciones límite y aumentar la importancia de dicho estado. Esta característica no será visible para el usuario.

- **Daño de arma**

Cada arma que el jugador puede empuñar cuenta con una magnitud de daño medida en formato de daño por segundo (DPS), que relaciona el daño que puede realizar cada arma por unidad de tiempo. Esto significa que las armas más fuertes harán más daño o dispararán más rápido. Para poder visualizar esto, el usuario tendrá disponible la magnitud en el área inicial de la nave.

En cuanto a las armas medievales, el jugador deberá usarlas contra los enemigos para estimar el daño real que pueden hacer. Esto aporta a la dinámica de juego en la que el usuario debe decidir bien qué arma lleva según los enemigos dejan caer las suyas.

- **Munición**

El número de disparos con cada arma que tiene disponible el usuario. Como sería demasiado sencillo enfrentarse con armas futuristas a enemigos con armas medievales, un método de equilibrio es implementar un número limitado de disparos. Una vez alcanzado dicho número, no se puede seguir utilizando el arma. Además, esta magnitud se equilibra entre las diferentes armas, pudiendo almacenar una cantidad menor según la cantidad de daño que cada arma puede realizar.

Es importante recalcar que el número máximo de disparos se divide entre el usuario y el arma. Las armas tienen un número de balas disponibles, que una vez gastadas deben ser recargadas hasta su número máximo. Por otro lado, el jugador tiene un número total de balas que se le otorgan al recoger el arma en la nave y equivalen a un número de recargas del arma. Las armas medievales no contarán con munición.

El número máximo de munición que puede llevar el usuario y la munición disponible en el arma se mostrarán numéricamente (como se explica en el apartado siguiente) y tendrá una retroalimentación por sonido cuando el usuario apriete el gatillo.

Las acciones que el usuario puede realizar son las siguientes:

- **Agarre**

La interacción base para todo el juego es el agarre. Esta es una acción natural e intuitiva con la que el usuario podrá familiarizarse rápidamente.

Servirá como forma principal de interactuar con el entorno, agarrando las diferentes armas para poder atacar.

- **Disparo**

Una interacción específica de las armas futuristas. Para realizarla se hará uso de los gatillos incorporados en los controladores de los cascos de VR de Meta. Esto hace que la interacción se dé de la forma más natural posible.

Esta acción permitirá al usuario disparar proyectiles desde las armas que realizarán daño a los enemigos al impactar con ellos. Junto a ese daño, los enemigos reproducirán un sonido de quejido que servirá como feedback sonoro para el usuario.

- **Ataque**

Las armas medievales no cuentan con disparos. Sin embargo, el usuario puede hacer uso de ellas para dañar a los enemigos por medio del movimiento. Al mover estas armas de forma rápida, las colisiones que generan con los cuerpos de los enemigos les quitarán vida.

- **Apuntado**

Al tratarse de un juego en VR, el apuntado se realizará de forma natural, moviendo las manos para acercarse la mira del arma. Esto ayudará a ver correctamente a dónde exactamente se está disparando el arma futurista escogida.

- **Recarga**

La recarga, al igual que el apuntado, se realizará de manera natural, cambiando un cargador por otro. Además, al insertar o agarrar el cargador, se reproducirá un sonido que imite los utilizados por otros videojuegos del estilo para alertar al jugador de ambas acciones.

- **Movimiento**

Aparte del agarre, la otra interacción básica que estará presente durante todo el juego es el movimiento. Este quedará implementado a través de los controladores de los cascos de VR de Meta. Concretamente se utilizarán ambas palancas (izquierda para movimiento y derecha para giro de la cámara). Junto al caminar vendrá un efecto sonoro de pasos que ayudará a la retroalimentación del jugador.

Las recompensas y objetos obtenibles que recibirá el usuario son los siguientes:

- **Puntuación**

Son la recompensa del juego, consiguiendo puntos según se derrotan enemigos. Cuanto más importante o fuerte es el enemigo, más puntos se consiguen. Estos puntos serán mostrados al jugador al terminar la partida.

- **Armas medievales**

Como se menciona arriba, los enemigos al morir podrán soltar armas medievales que el usuario podrá utilizar en combate contra los nuevos enemigos que aparezcan. El objetivo del usuario es recoger un arma para seguir con el combate una vez se le haya acabado la munición explicada anteriormente. Estas armas tendrán diferente cantidad de daño, dejando a

la elección del usuario qué arma llevar en cada momento.

6. Interfaz de Usuario (UI)

This is where you'll include a description of the user's control of the game. Think about which buttons on a device would be best suited for the game. Consider what the worst layout is, then ask yourself if your UI is still playable. A visual representation can be added where you relate the physical controls to the actions in the game. When designing the UI, it may be valuable to research quality control and user interface (UI) design information.

Para jugar al juego son necesarias unas gafas de VR, y todas las interacciones se gestionan a través de los controladores. Los controles se han especificado en el apartado "Player Control" del capítulo "Game Flowchart".

En cuanto a los elementos de UI en la pantalla, aparte del menú inicial y las pantallas de victoria y game over, el jugador tendrá varios elementos visibles en todo momento. El avatar llevará unas gafas militares que actuarán como HUD. A través de ellas el jugador podrá ver en todo momento las vidas restantes y la munición disponible. Las vidas se mostrarán en la esquina superior izquierda, como una barra de vida, con un icono de corazón al lado. La munición restante se mostrará en la esquina inferior derecha.

No se mostrará el temporizador en ningún momento.

Los audios serán acordes a cada situación, sin afectar a la presencia.

El peor escenario sería un usuario que no conoce la realidad virtual, pero el tutorial valdrá como familiarización con los controles y con el entorno

7. Diferencias de implementación (apartado grupal)

Durante el desarrollo de este trabajo, el equipo se ha encontrado con numerosos contratiempos y dificultades que han obligado a reducir el juego de lo expuesto en este documento a una demo de tiro en VR utilizando Unreal. Estos problemas se han centrado especialmente en torno a la utilización de dicho motor, siendo comprensible ya que la aplicación es el primer trabajo en él al que el equipo se ha enfrentado.

Cabe destacar que una de las mayores dificultades ha sido el intento de crear enemigos controlados por una inteligencia artificial a través de un árbol de comportamiento. El objetivo del área era implementar un enemigo base (del que surgirían nuevos en el futuro) que tuviese tres estados: buscando, persiguiendo y atacando. El flujo de estos estados empezaría con el enemigo buscando, moviéndose por puntos aleatorios del nav mesh, y pasando a perseguir al usuario si lo percibiese y estuviese demasiado lejos. También, si percibiese al enemigo y estuviese por debajo del umbral de distancia, realizaría un ataque y esperaría unos segundos.

Para esta sección, que finalmente no ha podido ser implementada, se llevó a cabo un periodo largo de investigación sobre el funcionamiento de esta arquitectura y componentes. Además, centraba la mayoría de su contenido en el desarrollo con C++ unido con blueprints ocasionales. Sin embargo, esta implementación en C++ ha probado ser más compleja de lo esperado en un inicio y no se ha conseguido que funcione en el tiempo establecido, llegando en muchos momentos a cerrar inesperadamente el editor de Unreal.

Como aspecto positivo, lo que sí se ha conseguido implementar en la aplicación de lo desarrollado en el documento en referencia a los enemigos es tanto un spawner como las dianas de entrenamiento (actualmente son los enemigos de la arena también). Por un lado, el spawner es una clase de C++ que, dados unos tiempos de espera inicial y cadencia de spawn, y una serie de puntos, crea enemigos con dicha cadencia en los puntos correspondientes. Por ahora, este spawner no ha podido ser usado en el juego principal ya que no contamos con rondas de enemigos. Y, por otro lado, las dianas se han implementado con blueprints, consiguiendo que puedan ser tanto fijas como móviles y que colisionen con las balas, desapareciendo al ser disparadas.

Otros cambios han estado relacionados con el player y sus armas. Por un lado, el player tendrá 5 vidas que se irán reduciendo cuando colisione con uno de los fuegos de la arena. Además, se han implementado 3 segundos de inmunidad donde no podrá recibir más daño. En cuanto al arma, al inicio se planteó que esta se moviera con el jugador entre escenas. Por falta de tiempo, cada escena contará con un arma en el aire que el jugador tendrá que utilizar. Por otro lado, la manera de interactuar con el UI será a través de pistolas específicas para ello. En el planteamiento inicial también se comentó que habría un sistema de recarga. Esto se deja para implementación futura. Otro de los cambios ha sido el movimiento del jugador. En la versión entregada se ha asignado un smooth move al joystick izquierdo, y un teleport (que en varias zonas no está configurado) y el snap turn al joystick derecho.

Finalmente, en cuanto al estilo artístico y visual se refiere, se ha querido marcar un contraste muy grande. Los dos primeros niveles, correspondientes a la nave y la sala de entrenamiento, presentan una estética futurista caracterizada por materiales metálicos, superficies tecnológicas, iluminación fría y elementos de ciencia ficción que transmiten una sensación de avance tecnológico y control. En contraste, el nivel final se desarrolla en un coliseo de inspiración medieval, donde predominan la piedra, las estructuras monumentales y una ambientación más cálida y orgánica, generando un fuerte contraste visual y narrativo respecto a los escenarios anteriores. Para potenciar la inmersión, se han utilizado sistemas de partículas de Unreal Engine mediante Niagara, empleados tanto en la creación de un portal con efectos energéticos dinámicos como en la simulación del fuego presente en el coliseo, aportando movimiento y riqueza visual a la escena. La iluminación también desempeña un papel fundamental en la identidad de cada entorno: en las zonas futuristas se ha optado por luces frías, tonos azulados y elementos emisivos que enfatizan el carácter tecnológico, mientras que en el coliseo predominan las luces cálidas procedentes de antorchas y braseros, creando una atmósfera más épica y acorde con la temática medieval.

Hemos utilizado dos modelos de pistolas: una que se empleará para seleccionar los botones de los canvas de UI, la cual contará con balas infinitas; y otra que será la que el jugador utilizará durante el desarrollo del juego, tanto en la sala de entrenamiento como en la misión final en el coliseo. En principio, el juego iba a contener una sala adicional que sería la sala de misiones; en ella únicamente habría habido un portal hacia la misión final, pero durante el transcurso del proyecto se decidió omitirla.

Los canvas también se han utilizado para mostrar la vida en el visor, de manera que el jugador sea consciente en todo momento de su vida restante. Lo mismo ocurre con las balas del arma, establecidas en 500, que aparecen en las pistolas para que el usuario pueda controlarlas en todo momento.

Por otro lado, el menú inicial cuenta con los botones de Play, Settings y Exit. La sección de Settings aún falta por implementarse. También falta un botón de Restart al perder o ganar, que permita volver a jugar; este elemento queda pendiente para ser desarrollado en la parte individual.

Por último, a nivel de sonido, se han creado audios narrativos relacionados con la historia. También se había planteado añadir sonidos de ambiente, como los pasos del usuario, el sonido del fuego en el coliseo, los sonidos de muerte del enemigo, etc.

8. Aporte individual

En este apartado explicaré mi aporte individual al proyecto. Cada apartado explicará un cambio o feature, y al final se comparará mi solución con la descrita en el GDD original.

8.1. Pasar blueprints a clases de C++

Lo primero que desarrollé en este apartado individual fue la creación de clases de C++. Estas clases se han creado tomando como referencia los blueprints del apartado grupal. Para su desarrollo, se han generado las clases desde *Tools>New C++ class*, se han implementado, y después se han asignado como padres del blueprint original. Las clases que se han desarrollado son las siguientes (con sus respectivos BP entre paréntesis): **ADamageSphere** (*BP_DamageSphere*), **ALevel2Manager** (*BP_Level2Manager*), **APistol** (*Pistol*), **APistolMenu** (*BP_APistolMenu*), **AProjectile** (*Projectile*) y **ATarget** (*BP_Target*).

Las funcionalidades casi no han variado.

De esta manera he conseguido aprender a crear clases de C++ desde blueprints, mejorando la complejidad del proyecto y creando una solución más robusta.

8.2. Mejorar aspectos visuales

Otra parte importante que creo que no terminamos de afinar en la parte grupal fue la parte visual. En esta entrega individual me he centrado en arreglar pequeños errores que afectaron bastante a la parte visual del juego.

Por un lado, he quitado los corazones de la pantalla (*WBP_Life*) en los niveles que no lo necesitaban. En otras palabras, he hecho que los corazones aparezcan solo en *Level2*. Para ello he editado el BP de *OlaiaVrPawn*: en el evento de *BeginPlay* he visibilizado el widget solo si el nombre del nivel es "Level2". Así he conseguido que el widget de corazones solo aparezca cuando tiene sentido.

Otro error que identifiqué fue que el valor del ammo que aparecía en la pistola no era el correcto. Este error pudo haber venido de la implementación de la clase de C++ *APistol*. El problema que encontré aquí fue que el texto del widget no se actualizaba correctamente desde el código, y lo arreglé creando una variable *CachedWidget* para guardar el objeto del widget en el script. Al final solo tuve que ajustar los *draw space* y los valores se empezaron a ver correctamente.

Otro pequeño bug que tenía la entrega anterior era que la mano izquierda tenía el modelo de la mano derecha, y si el jugador se miraba las manos veía 2 manos derechas, cosa que hace que la presencia se pierda. Para girarlo escalé la mano en el eje *y* con valor -1.

Por último, es importante que la *performance* del juego sea buena, y hasta ahora dejaba mucho que desear. Las mejoras que he hecho han sido en el **portal** y en el **fuego**, ya que estos son los modelos/elementos que más recursos utilizan. Por un lado, para mejorar el *performance* relacionado con el portal, he eliminado la colisión de cada partícula, ya que es innecesario en este juego (el portal tiene un *box collider* general). En cuanto a los fuegos, lo primero que hice fue eliminar algunos de la arena, dejando unos 5 fuegos. Además, después de analizar sus características, quité las colisiones de los *sparks* y el componente *Light* en *Flames*. Además, como última mejora, he eliminado prints innecesarios y revisado los eventos de *tick* de los blueprints, para comprobar que no había carga computacional innecesaria.

8.3. Añadir socket de arma en la muñeca del player

Otra mejora que he implementado ha sido un socket en la muñeca, donde se coloca la pistola durante todo el juego. Esto mejora la jugabilidad, ya que el player tendrá acceso a la pistola en todo momento, y no tendrá que ir físicamente a recogerla como se hacía hasta ahora. El usuario lo puede agarrar y utilizar con normalidad, y cuando lo suelta, la pistola vuelve al socket. Una característica importante es que la pistola que se instancia dependerá del nivel en el que se encuentre el jugador. En el primer nivel (*InitialShip*) la pistola será *BP_APistolMenu*, ya que no habrá límite de *ammo*. Por otro lado, en los niveles *Ship*, *GameOverShip* y *VictoryShip* no habrá ninguna pistola, y en los niveles *Tutorial* y *Level2*, la pistola será *Pistol*. Con este feature la gestión del arma es mucho más clara para el usuario, y se evitan problemas donde se pueda perder la pistola y el juego no se pueda continuar. Un pequeño *bug* que encontré cuando estaba desarrollando este apartado fue que cuando soltabas la pistola y presionabas el *trigger*, se disparaban balas. Para arreglarlo he creado una variable *bIsHeld*

que guarda si la pistola está siendo agarrada o no.

8.4. Nuevos enemigos + IA

Una nueva mejora de la app ha tenido que ver con la implementación de un sistema de Inteligencia Artificial como lo vimos en clase. Pero antes de explicar este sistema, es imprescindible comentar que se han añadido unos nuevos enemigos en la arena. Estos enemigos son unos pequeños monstruos morados, con sus animaciones, y que se spawnen en la arena cada x tiempo. Para hacer el sistema de spawn, he creado un nuevo blueprint llamado *BP_EnemyManager*, el cual instancia enemigos en la arena en unos targets específicos, cada X tiempo, y dependiendo del momento, añadirá 1, 2 o 3 enemigos en la escena.

Si el jugador se acerca a estos enemigos, a parte de hacerle perder una vida, este empezará a perseguirle. Para ello he implementado un sistema de IA como la de clase, con su respectivo *BT*, *AI Controller*, *Services*, *Decorators*, *Task*... Estos archivos se pueden encontrar dentro de la carpeta "*Olaia/IA*".

Una característica que he añadido a estos enemigos es que se pueden matar. Para ello he seguido los pasos de las dianas: se detecta la colisión y si el elemento colisionado es un *projectile*, se destruye.

8.5. Mejorar presencia

En los juegos de VR la presencia es lo que marca la diferencia. Para mejorar la experiencia del jugador en mi juego he cambiado varias cosas.

Primero, he cambiado la música de fondo, y la he adaptado a cada nivel. Por ejemplo, he añadido un audio base para las primeras escenas, pero en la arena he aplicado una música que genera más incertidumbre y tensión. Por último, he añadido un audio esperanzador en la escena de victoria, y uno más oscuro en la escena del game over.

Otro feature relacionado con los audios ha sido el sonido de los pasos. En el *tick* de *OlaiaVrPawn* he añadido la reproducción de un audio (el de un paso) cada vez que se desplace 110 unidades de su anterior posición. De esta manera he conseguido que el usuario tenga una mayor sensación de caminar, y que no parezca que está levitando.

Otro elemento que aporta a la presencia ha sido el uso de efecto hápticos. En mi caso, he añadido una vibración fuerte cuando el usuario recibe daño. Esto sumado al audio de "*auch!*" que ya tenía anteriormente, hace que el usuario entienda mucho mejor que está ocurriendo. Además, recordar que cuando se dispara también recibe una pequeña vibración en la mano que está siendo utilizada.

El *teleport* es un feature muy típico en realidad virtual, y aunque he tenido problemas para añadirlo en las escenas de las naves, sí que he podido añadirla en el nivel más importante: en la arena. El *navmesh* funciona correctamente aquí, y el usuario podrá escapar fácilmente de los enemigos de la arena.

Como último arreglo, he arreglado el *smooth move*. Al final he descartado la implementación original de *IA_SmoothMove*, ya que solo funcionaba en mi ordenador. Para arreglarlo he seguido los pasos de la clase de Realidad Virtual.

8.6. Diferencias con el GDD original

Mi producto final ha cambiado bastante desde la idea original. Por un lado, en la arena se ha reemplazado los monstruos por dianas (aunque al final sí se ha añadido una pequeña aparición de estos). Además, no hay un boss final: cuando eliminas todas las dianas se termina el juego. Por otro lado, no se ha añadido una ventana de ajustes como inicialmente se había mencionado, ni tampoco el uso de diferentes armas, recarga de armas, power ups o la interacción del usuario de saltar.

Aún así, tengo que decir que estoy orgullosa de mi trabajo, ya que el primer contacto con Unreal puede ser muy complejo, y aunque he tenido dificultades, he podido sacar el trabajo adelante.